

of a function. The `sqrt()` function is associated with `math`, which is a module, not a class.

Iterable vs. Iterator

Python's `for` loops are versatile. The statement `for i in range(3):` will run a block of code three times. The `range(3)` call isn't just Python's way of telling a `for` loop, "repeat some code three times." Calling `range(3)` returns a range object, just like calling `list('cat')` returns a list object. Both of these objects are examples of *iterableobjects* (or simply, *iterables*).

You use iterables in `for` loops. Enter the following into the interactive shell to see a `for` loop iterate over a range object and a list object:

```
>>> for i in range(3):
...     print(i) # body of the for loop
...
0
1
2
>>> for i in ['c', 'a', 't']:
...     print(i) # body of the for loop
...
c
a
t
```

Iterables also include all sequence types, such as range, list, tuple, and string objects, but also some container objects, such as dictionary, set, and file objects.

However, more is going on under the hood in these `for` loop examples. Behind the scenes, Python is calling the built-in `iter()` and `next()` functions for the `for` loop. When used in a `for` loop, *iterable* objects are passed to the built-in `iter()` function, which returns *iterator* objects. Although the iterable object contains the items, the iterator object keeps track of which item is next to be used in a loop. On each iteration of the loop, the iterator object is passed to the built-in `next()` function to return the next item in the iterable. We can call the `iter()` and `next()` functions manually to directly see how `for` loops work. Enter the following into the interactive shell to perform the same instructions as the previous loop example: